EURSKEM AI · ENGINEERING DOCUMENTATION

VOLUME 5

Node Type Engineering Standard

Why adding node types produces template and input/output errors, the contract every node type must satisfy, and the known-issues register with the remediation roadmap.

Platform: Agentic Workflow Platform

Version: 0.1.0 · Generated: 2026-08-22

Built by Ayush Khandelwal

Typed workflows compiled to LangGraph · zero-token preflight · strict evidence lifecycle

1. The problem

Adding a new node type has historically surfaced errors from two places: **template resolution** (downstream `{{node.field}}` references rejected by preflight) and **input/output mismatch** (configs or outputs that do not line up with what preflight, the Builder, and the generation LLM expect). Neither is a bug in the new node itself — both are the consequence of a node type being consumed by **six different systems**, only some of which update automatically when the type is registered.

2. Anatomy: the six consumers of a node type

#	Consumer	Mechanism	Auto-updates?
1	Runtime compiler	`discover_nodes()` imports every module in `app/nodes` and collects `@NodeRegistry.register` classes.	Yes
2	Builder palette & inspector	`NodeRegistry.manifest()` serves `GET /api/node-types`: config/input/output JSON schemas plus category, icon, family, execution kind and About.	Yes
3	Preflight template checks	Validates `{{node.field}}` references against `preflight_output_fields()`. The default exposes only static `output_schema` fields; **dynamic outputs need a per-node override or a typed-output builder in `logic_preflight.py`**.	Conditional — this is where TEMPLATE_UNKNOWN_OUTPUT_FIELD comes from
4	Preflight coverage gate	`tests/test_node_preflight_coverage.py` requires every new type to be acknowledged with a review note.	Manual by design
5	Generation & autofix prompts	The LLM catalog is built from the manifest, and the capability shortlist scores registry text — both auto-update. **But** the real-usage example embedded in the prompt is mined from workflows on disk (`about_synthesis.example_workflow_path`). A type used by no workflow ships with no example, so the model guesses config shapes.	Conditional — needs at least one reference workflow
6	Palette presentation tables	`categories.py` lookup tables and UI palette labels; sane defaults apply.	Yes (defaults)

> Conclusion: a new node type is only truly “wired in” when (a) its dynamic output fields are declared to preflight, and (b) at least one known-good workflow on disk exercises it, giving generation and autofix a real exemplar.

3. The node type contract (the standard)

Every node type is one module in `app/nodes/` containing one `@NodeRegistry.register` class. The complete surface is three Pydantic schemas and one async method:

Declaration	Purpose	Rule
`type_name`	Registry key and YAML `type:` value.	PascalCase, unique, stable forever.
`config_schema`	YAML config shape, validated at compile time.	Must set `extra="forbid"`; every field needs a description (feeds the Builder form and the LLM catalog).
`input_schema`	What the node reads from state.	Keep minimal; document optional inputs in field descriptions.
`output_schema`	What the node writes to `node_outputs[node_id]`; the compiler validates every run output against it.	If outputs are config-dependent or open-shaped, override `preflight_output_fields()`.
`run(state, resolved_config)`	Async execution; returns a dict conforming to `output_schema`.	Never call providers without finite timeouts; never swallow errors silently.
`required_services(config)`	Services needed at runtime (e.g. `llm`, `email`).	Preflight aggregates these into the workflow's service-availability check.
`preflight_output_fields(config)`	Valid dotted template-reference suffixes beyond the static schema.	Entries may be exact fields or dotted prefixes (`data.*`).
`about` / `family` / `execution_kind`	Author-facing About tab, palette grouping, automation-boundary badge.	Optional; `about_synthesis` fills gaps from schemas and real workflow adjacency.

4. Template resolution rules

Templates are a restricted dotted-path resolver (`app/runtime/templating.py`), not a general template engine: `{{inputs.x}}`, `{{variables.x}}`, `{{outputs.node_id.field.path}}`, optional trailing `?`, numeric list indices. Preflight enforces these codes — each maps to a specific authoring mistake:

Code	Meaning
`TEMPLATE_UNKNOWN_NODE`	Reference names a node that does not exist upstream.

`TEMPLATE_UNKNOWN_OUTPUT_FIELD`	Field path not covered by the referenced node's `preflight_output_fields()` — the most common error when a new node type has dynamic outputs.
`TEMPLATE_NOT_UPSTREAM`	Reference reaches for a node that is not upstream of the current one (would be a runtime race).
`TEMPLATE_NULLABLE_NESTED_ACCESS`	Path traverses a field whose declared type permits `None`; would crash mid-run.
`TEMPLATE_STATICALLY_EMPTY_FIELD`	Reference can only ever substitute a statically-known-empty value.
`TEMPLATE_UNKNOWN_INPUT` / `TEMPLATE_UNKNOWN_VARIABLE` / `TEMPLATE_UNKNOWN_STRUCTURED_FIELD`	Reference to an undeclared input, variable, or structured-output field.
`TEMPLATE_SELF_REFERENCE` / `TEMPLATE_UNBALANCED` / `TEMPLATE_CONDITIONAL_UPSTREAM`	Self reference; malformed braces; reference to a node that only executes on another branch.

5. Known issues register (current)

ID	Issue	Severity	Remediation
NT-1	Dynamic-output node types without a `preflight_output_fields()` override produce TEMPLATE_UNKNOWN_OUTPUT_FIELD for valid references.	High	Contract §3 makes the override mandatory for open-shaped outputs; the conformance harness detects untyped dynamic outputs automatically.
NT-2	New types appear in no workflow, so generation/autofix prompts carry no real-usage example and the LLM guesses config shapes.	High	Reference-workflow corpus (roadmap R2) gives every type curated exemplars.
NT-3	The `ACKNOWLEDGED_NODE_TYPES` snapshot fails on every new type with an opaque diff.	Medium	Replaced by a per-type review record plus executable conformance checks.
NT-4	Builder round-trip writes derived Start-node `inputs` back into saved YAML, mutating shipped workflows.	Medium	Fix in `yaml-bridge.ts`: emit only explicit inputs; the round-trip test enforces zero drift.
NT-5	Four shipped workflows fail `--warnings-as-errors` (nullable nested access and required-nullable extraction fields).	Medium	Per-workflow YAML fixes or explicit waivers.
NT-6	No executable per-type conformance battery exists today; mistakes surface only during authoring.	Medium	Conformance harness (roadmap R1).

6. Remediation roadmap

Step	Deliverable	Effect
R1	Node conformance harness: auto-parametrized tests over the whole registry (config strictness, manifest completeness, minimal-workflow compile + preflight, template matrix per declared output field, required-services validity, stub-executed output validation).	A new type is green only when the whole contract holds; failures become actionable instead of being discovered mid-authoring.
R2	Hidden reference corpus `workflows/reference/<NodeType>/*.yaml`: machine-gated example workflows (at least 7 per type, 400+ total), invisible in the UI library,	Every type gains curated exemplars; preflight gains a regression net; autofix and generation update automatically forever.

	scanned by `about_synthesis`, preflight CI, and generation exemplar selection.	
R3	Autofix/generation wiring: embed one or two reference snippets per shortlisted type in generate and repair prompts; regression test proving a freshly registered type needs zero manual edits anywhere.	The “autofix updates automatically” guarantee becomes a test.
R4	Node Type Studio (follow-up): UI-defined node types stored as declarative definitions (FieldSpec config + output schemas, prompt recipe), materialized into the registry behind the same contract, published only after passing R1 checks.	Node types can be added from the UI without code changes.

7. Checklist: adding a node type today

#	Step
1	Create `app/nodes/<name>.py` with the `@NodeRegistry.register` class; declare `type_name`, three schemas (`extra="forbid"` on config), and `run()`.
2	Declare `required_services()`; override `preflight_output_fields()` for any dynamic output shape (dotted prefixes allowed).
3	Add the review record in `tests/test_node_preflight_coverage.py` documenting which extension points were reviewed.
4	Write unit tests with `StubLLM`/fake services proving the output validates against `output_schema`.
5	Add at least one workflow using the node (reference corpus after R2) so generation and autofix get a real exemplar, then run `scripts/preflight_workflows.py --warnings-as-errors`.
6	Run the full backend test suite; the Builder palette and About tab pick the type up automatically via the manifest.